Software Architecture and Design Specification

Project: Student attendance management System

Version: 1.0

Authors: Chandana Lakya

Status: Sample

Revision History

Version	Date	Author	Change Summary
1.0	30/09/2025	Chandana Lakya	Initial draft prepared

Approvals

Role	Name	Signature
QA Lead	Chandana Lakya	
Test Engineer	Chitra Madarakhandi	
Dev Lead	Deesha C	
Product Owner	Daneshwari M	

1. Introduction

1.1 Purpose:-

Defines the architecture and design of the Student Attendance Management System (SAMS), supporting faculty attendance marking, student attendance tracking, and administrative reporting.

1.2 Scope

SAMS is a mobile App system for:

- Faculty marking attendance digitally
- Students viewing subject attendance
- Ensuring secure and reliable access

1.3 Audience

Faculty – to mark and review student attendance.

Administrators – to generate reports and manage records.

Students – to view their attendance status.

Developers – to implement and maintain the system.

1.4 Definition

- **SRS** → *What the system must do*
  - Defines **functional requirements (FRs)** → login, attendance marking, reports, audit logs.
  - Defines **non-functional requirements (NFRs)** → performance, scalability, availability.
  - Security requirements → TLS, AES encryption, RBAC, session timeout.
  - User roles → students, faculty, admins.
  - Example FR: "Faculty shall be able to mark attendance in real-time"
  - SRS_Template for SE
- **SAD** → *How the system will be designed & built*
  - Defines **architecture pattern (Layered)**, UML diagrams, API design.
  - Technology stack → Flutter frontend, Firebase/MySQL backend, OAuth2 authentication
  - SAD_Template.doc
  - Risks & mitigations → biometric fallback, caching for downtime.
  - Security model → STRIDE threat modeling (spoofing, tampering, DoS)
  - SAD_Template.doc
  - Example: REST API for marking attendance → /api/attendance/mark.
- **STP** → *How the system will be tested*
  - Defines **test items & features** (login, attendance marking, reports, alerts)
  - Test_Plan_ATM_Sample.docx
  - Test approach → unit, integration, system, UAT.
  - Security validation → penetration tests, session timeout, TLS validation.
  - Non-functional testing → load testing for 1000+ users.
  - Exit criteria → "No Critical/High severity defects open, RTM achieved"

2. Document Overview

2.1 How to use this document

SAD is a developer/tester reference: it maps SRS requirements to design modules, shows architecture decisions, provides API specs and sequence flows, states security design & threat mitigations, and

identifies acceptance criteria and test case links (RTM entries). Use it when implementing modules, writing tests, or doing architecture reviews.

2.2 Related Documents

SRS, STP, RTM.

3. Architecture

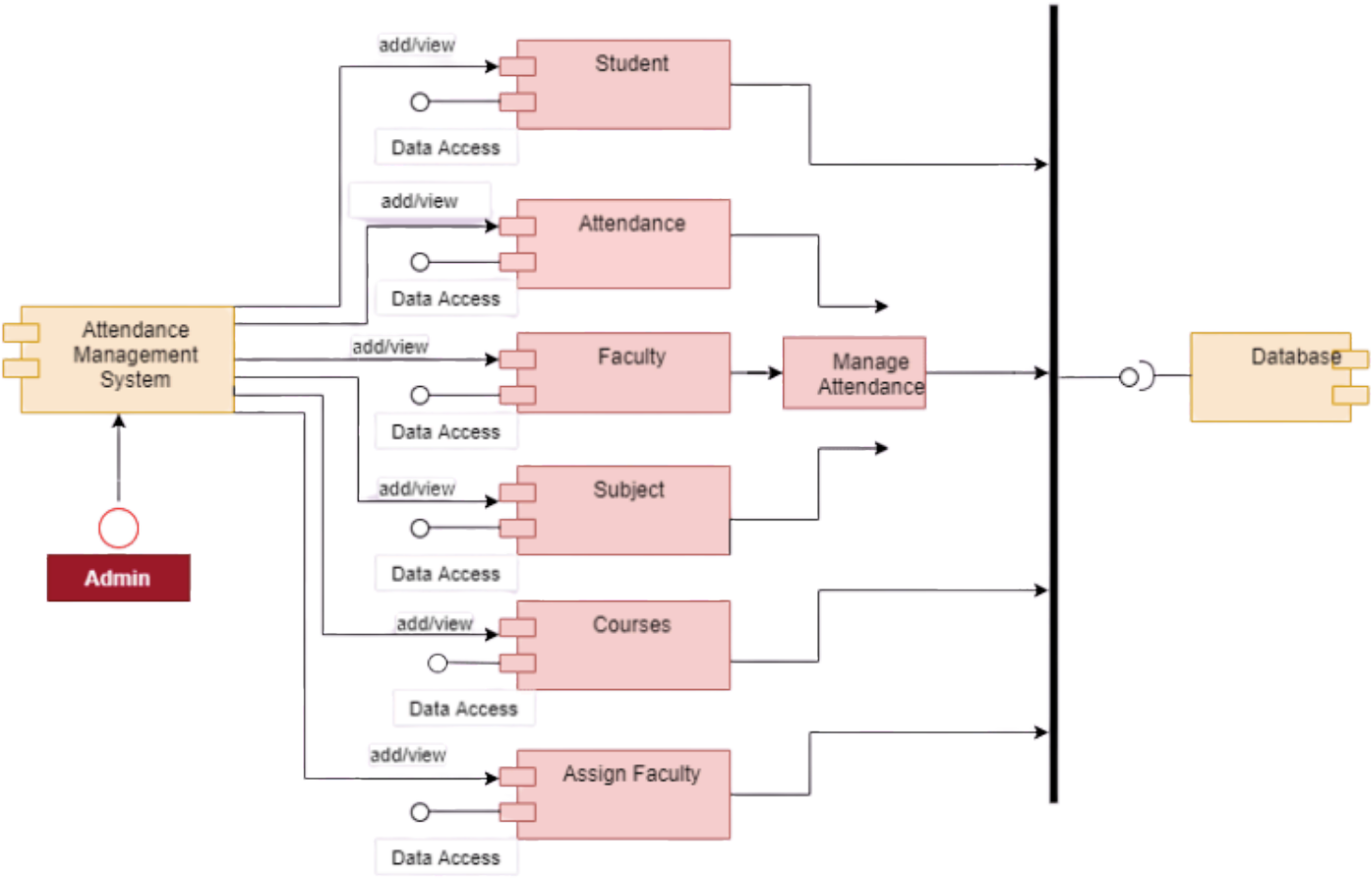
3.1 Goals & Constraints

- Goals:
  - Secure, accurate attendance tracking
  - Real-time data access and analytics
  - 99.5% system availability during semesters
- Constraints:
  - Institutional authentication integration
  - Support for Android, and web
  - Attendance edit permission limited to faculty
  - Data retention for at least 5 years

3.2 Stakeholders & Concerns

- Students: Quick mobile access, privacy, self-monitorable analytics
- Faculty: Simple marking/editing, justifications, download/export tools
- Admins: Audit logs, defaulter lists, suspicious activity alerts
- Developers: Modularity, REST APIs, clear interfaces, testability

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **Mobile/Web UI:** Flutter (mobile) and responsive PWA for web. Presents class lists, mark interface, student attendance views, and admin dashboards. Offline-first UI with local cache and retry.
- **Auth Service:** Integrates institutional SSO (OAuth2/SAML) + biometric plugin and enforces RBAC; issues JWT session tokens.
- **Attendance Service:** Core business logic to accept marks, validate duplicates, enforce class schedules, and require edit justification. Exposes REST endpoints (mark/edit/fetch).
- **Audit & Logging Service:** Appends immutable audit records (before/after values, user id, timestamp, justification); supports tamper-proof storage and admin queries.
- **Sync/Queue Service:** Handles offline marks (local queue, signed payload), retries and reconciliation on reconnect.
- **Database:** Primary store (PostgreSQL/MySQL) for attendance records, indices for fast queries, encrypted at rest.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is chosen for this project due to its clear separation of user interface, security, and persistence layers—exactly a match for the use-case and backend-service split shown in our diagrams. This supports maintainability, testability, and easier scalability. Microservices were rejected for this iteration as the project's domain and operational complexity don't require such granularity.

3.6 Technology Stack & Data Stores

- Front-End: Flutter (web/mobile), responsive HTML/CSS
- Backend: Node.js, or Python Flask REST APIs

- Database: MySQL/PostgreSQL
- Authentication: Institutional API, biometric (Fingerprint/FaceID plugins)
- Protocols: TLS 1.2+
- Testing Tools: Selenium WebDriver, Firebase Test Lab, Postman

3.7 Risks & Mitigations

Risk	Mitigation
Biometric failure	Rollback to password or institutional ID login
Sync issues	Offline cache, automatic retry on reconnection
Unauthorized edits	Immutable logs, role-based edit checks
Scalability	Firebase/Cloud functions, database sharding
Performance limits	Local cache, background sync, Database indexing, load-tested API endpoints,

3.8 Traceability to Requirements

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SAMS-F-001	User Authenticatio n(password/bi ometric)	4.1 / DS-Auth-01	Auth Module	TC-Auth-01	P	
SAMS-F-010	Faculty marks attendance	4.2 / DS-Att-01	Attendance Module	TC-Att-01	N	
SAMS-F-020	Student views sub-wise attendance	4.3/DS-Rep-01	Report Module	TC-Rep-01	P	
SAMS-F-030	Audit logs for all edits	4.4/DS-Log-01	Log Module	TC-Log-01	N	
SAMS-NF-001	Response time target	5 / DS-Perf-01	WebUI / CoreAPI	TC-Perf-01	N	

3.9 Security Architecture

Spoofing: institutional SSO + biometric + device validation.

Tampering: immutable audit logs and write patterns.

Repudiation: signed operations, logs with user id and justification.

Information Disclosure: TLS 1.2+ and AES-256 at rest.

DoS: rate limiting, API throttling, caching for spikes.

Privilege Escalation: enforce RBAC server-side. Session timeout 15 min inactivity

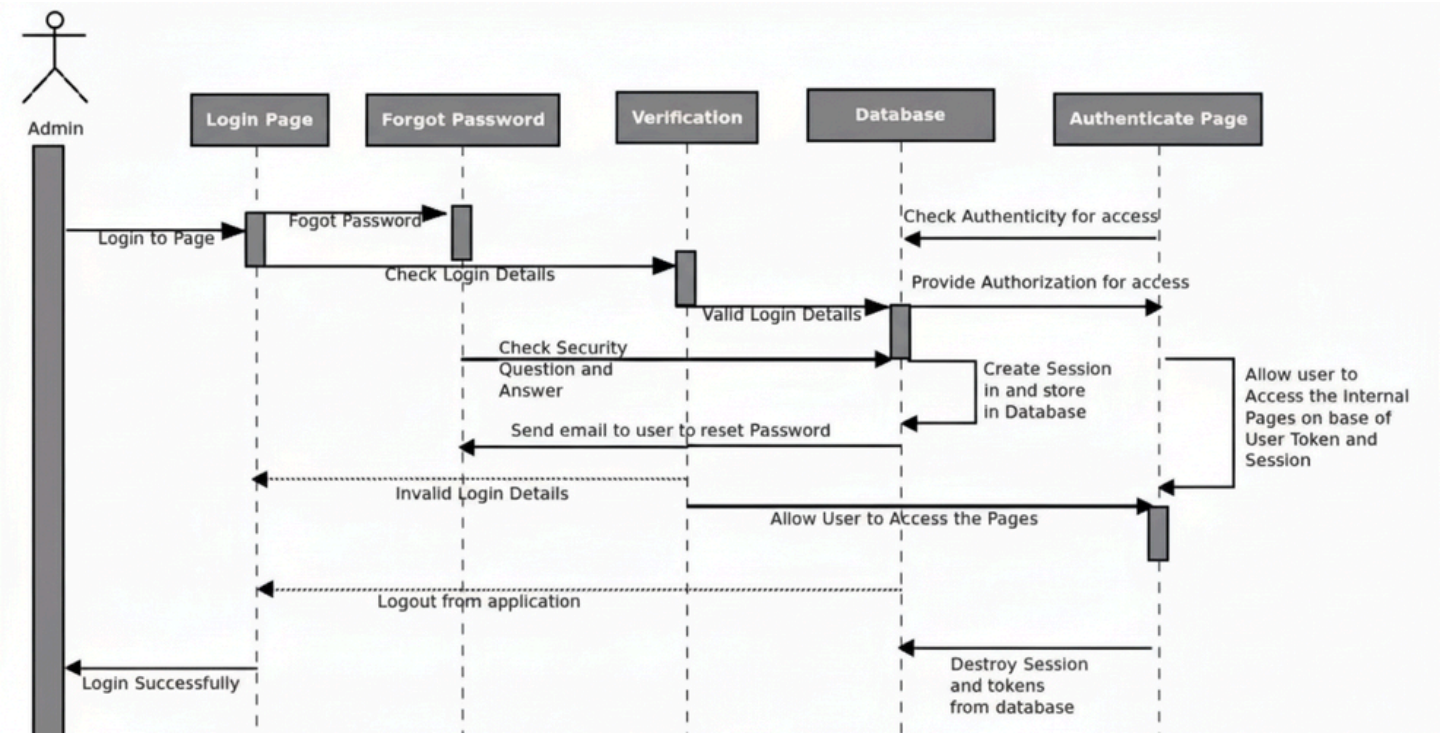
4. Design

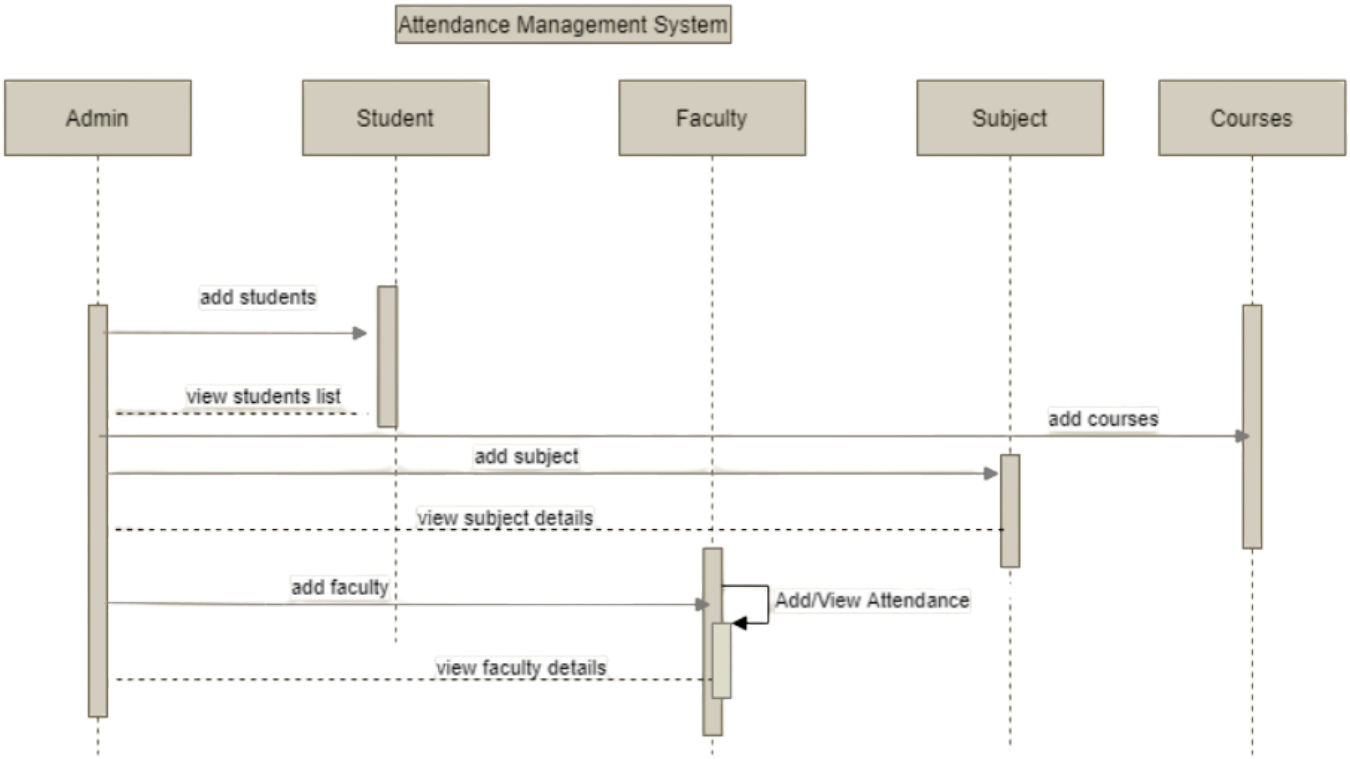
4.1 Design Overview

- Mobile-first UI, RESTful backend (stateless services), DB with indexes for attendance queries, asynchronous sync for offline marks, push/real-time updates for student views, and monitoring/alerts for suspicious activity and performance.

4.2 UML Sequence Diagrams

At least 2 sequence diagrams covering 2 flows specific to your project





4.3 API Design

POST /api/login

Request: { "username":"abc@uni.edu", "password":"xxx" }

Response 200: { "token":"<jwt>", "role":"faculty", "expires":"2025-10-01T10:00:00Z" }

POST /api/attendance/mark

Request: { "faculty_id":123, "class_id":789, "date":"2025-09-29", "students":[{"id":1,"present":true}, {"id":2,"present":false}], "token":"<jwt>" }

Response: { "status":"ok","timestamp":"2025-09-29T09:12:34Z" }

PUT /api/attendance/edit

Request: { "faculty_id":123, "record_id":555, "present":false, "justification":"missed class due to activity", "token":"<jwt>" }

Response: { "status":"ok","audit_id":987 } (edit logged immutably).

GET /api/reports/attendance?student_id=1&from=2025-09-01&to=2025-09-30

Response: { "student_id":1, "course_scores":[{"course":"CS101","percentage":86.5}], "daywise":[{"date":"2025-09-01","present":true}, ...] }

GET /api/admin/defaulters?threshold=75&course=CS101

Response: CSV or JSON list of students below threshold.

GET /api/audit/logs?record_id=555

Requires admin role; returns before/after + user + timestamp + justification.

4.4 Error Handling, Logging & Monitoring

Errors: Use standard HTTP codes (400, 401, 403, 404, 409, 500). Present user-friendly messages on UI without revealing stack traces. Log full error details server-side for diagnostics.

Logging: Structured logs (JSON) with request id, user id, timestamp, action, and outcome. Audit logs store before/after attendance values, justification and are write-once (append only) and retained per retention policy.

Monitoring: Track uptime, request latency, error rates, queue sizes and suspicious login attempts. Set alerts: for error rate sustained for 5 min, login lockout events trigger security alert. Validate in STP with security tests and load tests.

4.5 UX Design

- Mobile-first layout; large tap targets for faculty marking; quick "mark all present" shortcut; visible undo for last few seconds; student dashboard shows overall %, course drilldown and day-wise list; accessible color contrast and screen-reader compatibility; offline indicator and sync status. Provide sample screens in appendix

4.6 Open Issues & Next Steps

- Suggested next steps: generate UML diagrams and embed in SAD, implement OpenAPI spec, decide DB engine (Firestore vs. Postgres), finalize biometric SDK vendor, implement push notifications for defaulter alerts (optional), and schedule penetration & load tests per STP. Document the RTM fully mapping all SRS IDs to design & tests.

5. Appendices

5.1 Glossary:

- **SAMS:** Student Attendance Management System.
- **TLS:** Transport Layer Security (in transit encryption).
- **RBAC:** Role Based Access Control.
- **Audit Log:** Tamper-resistant record of changes.
- **Defaulter List:** Students below configured attendance threshold.

References: SRS, STP, IEEE-42010, OWASP, NIST-SP-800-160